Adding a Filter

For reasons that are utterly opaque, we'd like certain strings to contain a cat, so we'll add an `|addcat` filter to achieve this. First, let's build the filter object:

```
// index.php
require_once __DIR__ . '/vendor/autoload.php';

$loader = new \Twig\Loader\FilesystemLoader(__DIR__ . '/views');
$twig = new \Twig\Environment($loader);

$filter = new \Twig\TwigFilter('addcat', function ($string) {
    return $string . "... and Jess the cat. Meow.";
});
```

Next, we register that filter object with Twig before rendering…

```
$twig->addFilter($filter);
print $twig->render('add-cat.html.twig');
```

… now we can make use of the `|addcat` filter in the rendered template (and of course any other template that is extended, used, or included; the filter is available globally in that Twig environment):

```
{# views/add-cat.html.twig #}
<div>
    {{ 'Anna, Paul'|addcat }}
</div>
$ curl -v http://localhost:8080/index.php
<div>
    Anna, Paul... and Jess the cat. Meow.
</div>
```

We've chosen to work with string data here, but a filter could also be applied to an object, depending on the Twig expression and underlying PHP data type. For example, a context variable that is bound to a `DateTime` instance would pass the object to the filter callable when used. In the filter expression, everything on the left-hand side of the `|` pipe character expression is evaluated and passed as an argument to the anonymous function we used to construct the `Twig\TwigFilter` object. The data arriving at the callback can be any PHP type. Alternative methods of creating a Twig filter object callable are: